Machine Translated by Google

[Chap 14] Basics of Natural Language Processing

Machine Translated by Google

outline

- Language-based data, such as news and customer reviews, often contains information important to

  business – Language data is unstructured data expressed as large amounts of text – A representative type of Big Data

- Natural language data

  – Data consisting of everyday language expressions

- NLP (Natural Language Processing) – A collective

  term for technologies for the analysis and processing of natural language

Machine Translated by Google

BOW model

- The first step in processing natural language with a computer is to convert the language into numbers

  It is a dream

- Change the following sentence into numbers?

I highly recommend this movie. It's really different from other movies.

- Frequency of words in the above sentence

〈표 14-1〉 리뷰 문장의 빈도 벡터

이	영화	완전	강추	진짜	다른
1	2	1	1	1	1

Machine Translated by Google

BOW Model (continued)

- BOW (Bag Of Words) Model – A model

  that converts words appearing in a sentence or document into numbers by counting only their frequency,

  ignoring their order –

  The BOW model for the previous sentence becomes (1, 2, 1, 1, 1, 1).

[그림 14-1] BOW 모델의 개념

Machine Translated by Google

Practical Exercise on Building a BOW Model

- Example of a long document: The 6th National Informationization Basic Plan
  - Available for download from the NIA website (www.nia.or.kr) – Save as plain text • Set text encoding to 'UTF-8'

• Read file

```
f = open('정보화기본계획.txt', 'r', encoding='utf8')
texts = []
while True:
    line = f.readline()
    if not line: break
    texts.append(line)
text = ' '.join(texts)
print(text)
```

```
지능정보사회 구현을 위한
제6차 국가정보화 기본계획(2018~2022)


01

H

추진배경 :
초연결 지능화
패러다임 전환

CHAPTER

01

추진배경
초연결 지능화 패러다임 전환

CHAPTER

  모든 것이 서로 연결되고, 보다 지능화된 
● 인터넷과 컴퓨터 기반의 '정보화'사회를 넘
능화' 사회로 빠르게 진입 ...
```

Machine Translated by Google

Text Preprocessing

- Separate text into words
  - Generally, use a morphological analyzer to extract the base forms of nouns or adjectives, etc. •
    Korean morphological analyzers: Use the konlpy library or Khaii, etc. – Here,
    we will practice by simply separating by spaces.

- Text prepreprocessing for word separation – Word separation in Korean text: spaces,
  newline characters (\n), periods, question marks, exclamation marks, commas, hyphens (-), colons (:), tildes
  (ÿ), slashes (/), etc. – In addition, there are various
  punctuation marks such as parentheses, quotation marks, arrows, and reference marks (ÿ), and in official documents
  The middle dot (·) and various bullet characters (ÿ, •, ÿ, etc.) are also used.
  – Replace all word separator characters with spaces

```python
special = ['\n', '.', ',', '-', ':', '~', '/', '·', '●', '▲', '※',
           '(', ')', '[', ']', '“', '”', '→']
for c in special:
    text = text.replace(c, ' ')
print(text)
```

```
지능정보사회 구현을 위한  제6차 국가정보화 기본계획 2018 2022     01   H   추진배경   초연결 지능화  패러다임 전환   CHAPTER   01   추진배경  초연결 지능화 패러다임 전환   CHAPTER     모든 것이 서로 연결 되고  보다 지능화된 사회로 진입    인터넷과 컴퓨터 기반의  정보화 사회를 넘어  전 세계는  초연결 Hyper Connectivity  지능화  사회로 빠르게 진입 ...
```

Machine Translated by Google

Calculate word frequency

- Split text based on spaces – split(' ') :

  Creates a list of tokens split based on spaces

```
wordList = text.split(' ')
print(wordList)
```

```
['지능정보사회', '구현을', '위한', '', '', '제6차', '국가정보화', '기본계획', '2018', '2022',
'', '', '', '', '', '', '01', '', '', '', 'H', '', '', '', '추진배경', '', '', '', '초연결', '지능
화', '', '', '패러다임', '전환', '', '', '', 'CHAPTER', '', '', '', '01', '', '', '', '추진배경',
'', '초연결', '지능화', '패러다임', '전환', '', '', '', 'CHAPTER', '', '', '', '', '', '모든', '것
이', '서로', '연결되고', '', '보다', '지능화된', '사회로', '진입', '', '', '', '', '인터넷과', '컴
퓨터', '기반의', '', '정보화', '사회를', '넘어', '', '전', '세계는', '', '초연결', 'Hyper', ...
```

- Calculation of frequency per

  token – Using dictionary data type •
  Stored in (key, value) format

```
1 words = {}
2 for w in wordList:
3     if len(w)==0: continue
4     if w not in words: words[w] = 1
5     else: words[w] += 1
6 print(words)
```

(Line 1) Create an empty dictionary (Line 2) Store each word in the document in w (Line 3) Skip to w if it is a word of length 0 (Line 4) If w appears for the first time in the dictionary 'words', register the word as a new key and set its value to 1 (Line 5) If w has appeared before, increment the value of that word

```
{'지능정보사회': 30, '구현을': 9, '위한': 247, '제6차': 8, '국가정보화': 30, '기본계획': 16,
'2018': 9, '2022': 8, '01': 3, 'H': 19, '추진배경': 2, '초연결': 20, '지능화': 90, '패러다임':
5, '전환': 24, 'CHAPTER': 13, '모든': 12, '것이': 1, ...
```

Machine Translated by Google

Sorting and Stopword Processing

- Sort the words in the dictionary in descending order of frequency

```
sortedWords = sorted(words.items(), key=(lambda x: x[1]), reverse=True)
print(sortedWords)
```

```
[('등', 444), ('및', 431), ('위한', 247), ('구축', 197), ('데이터', 163), ('지원', 148), ('기반', 148), ('추진', 136), ('개발', 135), ('서비스', 128), ('기술', 107), ('확대', 106), ('통해', 105), ('과기정통부', 105), ('강화', 104), ('디지털', 94), ('등을', 92), ('지능화', 90), ('지능형', 86), ('빅데이터', 85), ('통한', 84), ('활용', 83), ('1', 82), ('AI', 74), ('위해', 74), ('수', 73), ('2', 72), ('조성', 72), ('마련', 71), ('클라우드', 71), ('17', 68), ('3', 67), ('확산', 66), ('4차', 64), ('산업', 63), ('IoT', 63), ('교육', 62), (' 맞춤형', 58), ('18', 56), ('스마트', 56), ('분야', 54), ('플랫폼', 51), ('인공지능', 50), ('국가', 48), ('네트워크', 47), ('4', 47), ('분석', 46), ('있는', 46), ('필요', 45), ('글로벌', 44), ('인프라', 44), ('ICT', 43), ('대한', 43), ('16', 42), ('제고', 42), ('활성화', 42), ...
```

  – Common words that always appear in sentences regardless of topic, such as 'etc.', 'and', and 'for'—in other words, stopwords—occupy high frequencies and need to be removed.

- Added stop word removal section

```
stopwords=['등', '및', '위한', '통해', '등을', '통한', '활용', '위해',
    '수', '있는', '필요', '대한', '관련', '있도록', '다양한',
    '1', '2', '3', '4', '5', '6', '7', '8', '9', '0',
    '13', '14', '15', '16', '17', '18', '19', '20', '21', '22']
words = {}
for w in wordList:
    if len(w)==0: continue
    if w in stopwords: continue
    if w not in words: words[w] = 1
    else: words[w] += 1

sortedWords = sorted(words.items(), key=(lambda x: x[1]), reverse=True)
print(sortedWords)
```

```
[('구축', 197), ('데이터', 163), ('지원', 148), ('기반', 148), ('추진', 136), ('개발', 135), ('서비스', 128), ('기술', 107), ('확대', 106), ('과기정통부', 105), ('강화', 104), ('디지털', 94), ('지능화', 90), ('지능형', 86), ('빅데이터', 85), ('AI', 74), ('조성', 72), ('마련', 71), ('클라우드', 71), ('확산', 66), ('4차', 64), ('산업', 63), ('IoT', 63), ('교육', 62), ('맞춤형', 58), ('스마트', 56), ('분야', 54), ('플랫폼', 51), ('인공지능', 50), ...
```

Machine Translated by Google

BOW Visualization Practice

- Install wordcloud package

```
C:\WINDOWS\system32>pip install wordcloud_
```

– If the following error occurs, additional installation of Microsoft Visual C++ Build Tools is required.

```
error: Microsoft Visual C++ 14.0 is required. Get it with "Microsoft Visual C++ Build
Tools": https://visualstudio.microsoft.com/downloads/
```

ÿ Install the Community Version from https://visualstudio.microsoft.com/ko/downloads

- Font

Installation – The file 'KoPubWorld Dotum Medium.ttf' of the public font 'Kopub World Dotum M5

Copy together to the folder containing the code

- https://gongu.copyright.or.kr/gongu/main/main.do

Machine Translated by Google

word cloud

- Word Cloud Generation (Full Code: ex14-2.py)

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

wordcloud = WordCloud(font_path='./KoPubWorld Dotum Medium.ttf',
    background_color='white', width=1024, height=1024,
    max_font_size=100, min_font_size=10, max_words=300,
    colormap='Dark2').generate_from_frequencies(words)
plt.figure()
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()
```

Machine Translated by Google

Meaning by Option

〈표 14-2〉 워드클라우드의 옵션별 의미

옵션명	의미
font_path	워드클라우드에 사용할 폰트파일
background_color	워드클라우드 배경색
width	워드클라우드 가로크기 (픽셀 수)
height	워드클라우드 세로크기 (픽셀 수)
max_font_size	가장 큰 글자 크기
min_font_size	가장 작은 글자 크기
max_words	워드클라우드에 나타낼 최대 단어 갯수
colormap	워드클라우드에 사용할 색상표 이름1

Machine Translated by Google

Word cloud visualization results

You can intuitively grasp the central theme of this document by looking at the visualized figure.

– The focus of the policies described in the document appears to be data (big data), cloud, artificial intelligence, etc.

Machine Translated by Google

Frequency vectors and DTM

- Measure similarity between the following 3 sentences

문장 A: 이 영화 완전 강추 진짜 다른 영화

문장 B: 진짜 강추 완전 좋은 영화

문장 C: 자막 진짜 엉망

- Document-Term Matrix (DTM) – Integrates the BOWs of all sentences A, B, and C

〈표 14-3〉 리뷰 문장들의 DTM

단어	이	영화	완전	강추	진짜	다른	좋은	자막	엉망
문장 A	1	2	1	1	1	1	0	0	0
문장 B	0	1	1	1	1	0	1	0	0
문장 C	0	0	0	0	1	0	0	1	1

Machine Translated by Google

Vector dot product and sentence similarity

- Vector inner product

$$\boldsymbol{x} \cdot \boldsymbol{y} = x_1 y_1 + x_2 y_2 + x_3 y_3 = |\boldsymbol{x}||\boldsymbol{y}| \cos(\theta) \qquad \text{(식 14–1)}$$

  – The more similar the directions of two vectors from the origin, the larger their inner product; if they are orthogonal to each other with respect to the origin, the inner product is 0, and if they point in opposite directions, the inner product is a negative value.

- Dot product between sentences A, B, and C

$$\boldsymbol{a} \cdot \boldsymbol{b} = (1\times0)+(2\times1)+(1\times1)+(1\times1)+(1\times1)+(1\times0)+(0\times1)+(0\times0)+(0\times0) = 5 \qquad \text{(식 14–2)}$$

$$\boldsymbol{a} \cdot \boldsymbol{c} = (1\times0)+(2\times0)+(1\times0)+(1\times0)+(1\times1)+(1\times0)+(0\times0)+(0\times1)+(0\times1) = 1$$

  – Reflects that sentence A has high similarity to B but low similarity to C

- Simple internal problem

문장 C′: 자막 진짜 엉망 자막 진짜 엉망 자막 진짜 엉망 자막 진짜 엉망 자막 진짜
엉망 자막 진짜 엉망

  – Semantically, C and C' are identical, but $c' = (0,0,0,0,6,0,0,6,6)$ since $\boldsymbol{a} \cdot \boldsymbol{c}' = 6$ This becomes

Machine Translated by Google

Cosine similarity

- Cosine similarity

$$similarity(\boldsymbol{x}, \boldsymbol{y}) = \cos(\theta) = \frac{\boldsymbol{x} \cdot \boldsymbol{y}}{|\boldsymbol{x}||\boldsymbol{y}|} \qquad \text{(식 14–3)}$$

– The value obtained by dividing the dot product of vectors by the absolute value

of each vector – Enables the measurement of word occurrence pattern similarity while excluding the influence of document length

- Cosine similarity between sentence A and sentences B and C'

$$similarity(\boldsymbol{a}, \boldsymbol{b}) = \frac{\boldsymbol{a} \cdot \boldsymbol{b}}{|a||b|} = \frac{5}{3 \times 2.24} = 0.75$$

$$similarity(\boldsymbol{a}, \boldsymbol{c}') = \frac{\boldsymbol{a} \cdot \boldsymbol{c}'}{|a||c'|} = \frac{6}{3 \times 10.39} = 0.19 \qquad \text{(식 14–5)}$$

– Provides a value corresponding to semantic similarity regardless of document length

Machine Translated by Google

Limitations of cosine similarity

- Consider the following 3 sentences

문장 A: 이 영화 완전 강추 진짜 다른 영화

문장 B: 진짜 강추 완전 좋은 영화

문장 C: 이 영화 영화 자막 완전 진짜 완전 엉망

- The problem of cosine similarity

〈표 14-4〉 수정된 리뷰 문장들의 DTM

단어	이	영화	완전	강추	진짜	다른	좋은	자막	엉망
문장 A	1	2	1	1	1	1	0	0	0
문장 B	0	1	1	1	1	0	1	0	0
문장 C	1	2	2	0	1	0	0	1	1

$$similarity(\boldsymbol{a}, \boldsymbol{b}) = \frac{\boldsymbol{a} \cdot \boldsymbol{b}}{|a||b|} = \frac{5}{3 \times 2.23} = 0.75$$

$$similarity(\boldsymbol{a}, \boldsymbol{c}) = \frac{\boldsymbol{a} \cdot \boldsymbol{c}}{|a||c|} = \frac{8}{3 \times 3.46} = 0.77$$

– Although sentences A and C are semantically opposite, their cosine similarity is actually higher. –

Words such as 'movie,' 'completely,' and 'real,' which contributed to the high similarity between sentences A and C, are common words appearing in all reviews and are difficult to regard as containing meaningful information.

Machine Translated by Google

Concept of TF-IDF

- TF-IDF(Term Frequency-Inverse Document Frequency)

$$TF-IDF(w,d,D) = tf(w,d) \times idf(w,D) = tf(w,d) \times \log\left(\frac{N}{df(w,D)}\right)$$

  – A concept of dividing word frequency by the frequency of documents in which the word appears (log value) – Words that appear commonly across multiple documents have a lower TF-IDF value.

- TF-IDF of 'complete' in Document A

  – Since TF = 1 and DF = 3,

$$TF-IDF(\text{'완전'}, A, D) = tf(\text{'완전'}, D) = 1 \times \log\left(\frac{3}{3}\right) = 0$$

  – In other words, the word 'complete' reflects a lack of information.

Machine Translated by Google

Calculation of TF-IDF

- A function that calculates the frequency of word w within document d

```
def tf(w, d):
    cnt = 0
    for t in d:
        if t==w: cnt+=1
    return cnt
```

- Function to calculate the number of documents in which a specific word w appears (D: list of documents)

```
def df(w, D):
    cnt = 0
    for sent in D:
        if w in sent: cnt+=1
    if cnt==0: cnt = 1
    return cnt
```

Machine Translated by Google

Calculation of TF-IDF (continued)

- Function to calculate TF-IDF

```
def tfidf(w,d,D):
    N=len(D)
    return tf(w,d)*np.log(N/df(w,D))
```

- Calculate TF-IDF values for each word in sentences A, B, and C

```
sent_A = '이 영화 완전 강추 진짜 다른 영화'.split(' ')
sent_B = '진짜 강추 완전 좋은 영화'.split(' ')
sent_C = '이 영화 영화 자막 완전 진짜 완전 엉망'.split(' ')
corpus=[sent_A, sent_B, sent_C]
print(corpus)

print('<Sentence A>')
for w in sent_A:
    print(w, tfidf(w,sent_A,corpus))
print('<Sentence B>')
for w in sent_B:
    print(w, tfidf(w,sent_B,corpus))
print('<Sentence C>')
for w in sent_C:
    print(w, tfidf(w,sent_C,corpus))
```

Machine Translated by Google

Calculation of TF-IDF (continued)

- TF-IDF matrix for the entire document

〈표 14–5〉 단어별 TF–IDF 값

문서	이	영화	완전	강추	진짜	다른	좋은	자막	엉망
A	0.41	0	0	0.41	0	1.10	0	0	0
B	0	0	0	0.41	0	0	1.10	0	0
C	0.41	0	0	0	0	0	0	1.10	1.10

- Cosine similarity of TF-IDF vectors

$$\boldsymbol{a}' = (0.41, 0, 0, 0.41, 0, 1.10, 0, 0, 0)$$
$$\boldsymbol{b}' = (0, 0, 0, 0.41, 0, 0, 1.10, 0, 0)$$
$$\boldsymbol{c}' = (0.41, 0, 0, 0, 0, 0, 0, 1.10, 1.10)$$

$$similarity(\boldsymbol{a}', \boldsymbol{b}') = \frac{\boldsymbol{a}' \cdot \boldsymbol{b}'}{|a'||b'|} = \frac{0.1644}{1.24 \times 1.17} = 0.11$$

$$similarity(\boldsymbol{a}', \boldsymbol{c}') = \frac{\boldsymbol{a}' \cdot \boldsymbol{c}'}{|a'||c'|} = \frac{0.1644}{1.24 \times 1.61} = 0.08$$

– Just like the actual meaning, the similarity between A and B appears higher.

Machine Translated by Google

Calculation of TF-IDF (continued)

- Prepare TF-IDF data

```
wordSet = ['이','영화','완전','강추','진짜','다른','좋은','자막','엉망']
tfidf_a=[]
tfidf_b=[]
tfidf_c=[]
for w in wordSet: tfidf_a.append(tfidf(w,sent_A,corpus))
for w in wordSet: tfidf_b.append(tfidf(w,sent_B,corpus))
for w in wordSet: tfidf_c.append(tfidf(w,sent_C,corpus))

print('tfidf_a = ', tfidf_a)
print('tfidf_b = ', tfidf_b)
print('tfidf_c = ', tfidf_c)
```

```
[0.4054651081081644, 0.0, 0.0, 0.4054651081081644, 0.0, 1.0986122886681098, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 0.4054651081081644, 0.0, 0.0, 1.0986122886681098, 0.0, 0.0]
[0.4054651081081644, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0986122886681098, 1.0986122886681098]
```

Machine Translated by Google

Calculation of TF-IDF (continued)

- Calculate the absolute value of a vector

```
def magnitude(v):
    m=0
    for x in v: m += x**2
    return (np.sqrt(m))
```

- Calculate cosine similarity between vectors

```
def similarity(a,b):
    dot = np.dot(a,b)
    ma = magnitude(a)
    mb = magnitude(b)
    return(dot/(ma*mb))
```

- Calculate similarity between TF-IDF vectors of documents A, B, and C

```
print('Magnitude of tfidf_a = ', magnitude(tfidf_a))
print('Magnitude of tfidf_b = ', magnitude(tfidf_b))
print('Magnitude of tfidf_c = ', magnitude(tfidf_c))
print('similarity(tfidf_a, tfidf_b) = ', similarity(tfidf_a, tfidf_b))
print('similarity(tfidf_a, tfidf_b) = ', similarity(tfidf_a, tfidf_c))
```

Full code: ex14-4.py

Machine Translated by Google

Sentiment analysis

(SENTIMENT ANALYSIS)

Machine Translated by Google

Sentiment Analysis

- Understanding emotions expressed in natural language texts

- Diverse Applications –

  Understanding investor sentiment regarding specific stocks through stock bulletin boards, etc. – Market sentiment toward specific companies as portrayed in the media – Customer reactions to products or services – Checking the status of software or machinery through log files, etc.

- Types of Sentiment Analysis Techniques

  – Statistical methods •

    Positive/Negative dictionary-based • SO-PMI (Semantic-Oriented Pointwise Mutual Information)

  – Artificial neural network based

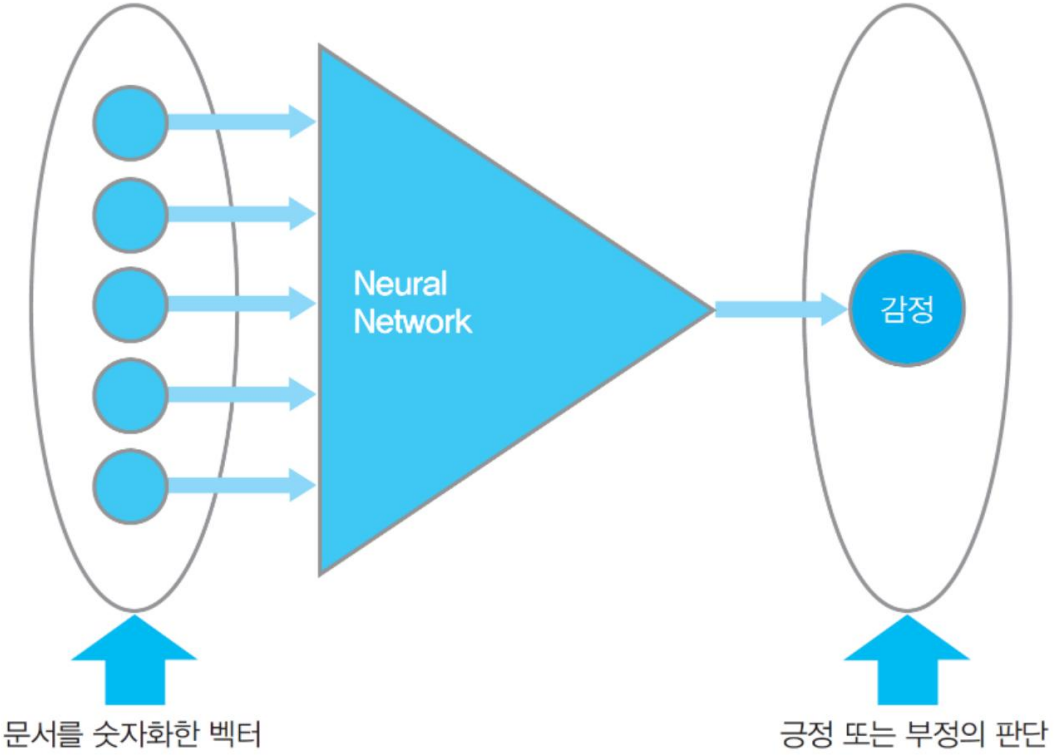
Machine Translated by Google

Sentiment analysis using artificial neural networks

- Neural Network Structure

  – Input: A vector converting the document into a number using tools like BOW (e.g., BOW) –

  Output: Representing positive/negative sentiment as 1 and 0



[그림 14–3] 감성분석을 위한 인공신경망 구조

Machine Translated by Google

IMDB movie review data

- As movie review data, it contains review content with words converted into numbers, along with a correct answer indicating whether the review is positive or negative – words are represented as numbers, and the word numbers are assigned in order of frequency of occurrence.

예제 ex14-5.py

```python
from tensorflow import keras
from tensorflow.keras.datasets import imdb
import numpy as np

(train_data, train_labels), (test_data, test_labels) = imdb.load_data()
print(type(train_data))

print(train_data.shape, train_labels.shape)
print(test_data.shape, test_labels.shape)
print('Train data 0 = ', train_data[0])
print('Train label 0 = ', train_labels[0])
```

```
Train data 0 =  [1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4,
173, 36, 256, 5, 25, 100, 43, 838, 112, 50, 670, 22665, 9, 35, 480, 284, 5, 150, 4,
172, 112, 167, 21631, 336, 385, 39, 4, 172, 4536, 1111, 17, 546, 38, 13, 447, 4, 192,
50, 16, 6, 147, 2025, 19, 14, 22, 4, 1920, 4613, 469, 4, 22, 71, 87, 12, 16, 43, 530,
38, 76, 15, 13, 1247, 4, 22, 17, 515, 17, 12, 16, 626, 18, 19193, 5, 62, 386, 12, 8,
316, 8, 106, 5, 4, 2223, 5244, 16, 480, 66, 3785, 33, 4, 130, 12, 16, 38, 619, 5, 25,
124, 51, 36, 135, 48, 25, 1415, 33, 6, 22, 12, 215, 28, 77, 52, 5, 14, 407, 16, 82,
10311, 8, 4, 107, 117, 5952, 15, 256, 4, 31050, 7, 3766, 5, 723, 36, 71, 43, 530, 476,
26, 400, 317, 46, 7, 4, 12118, 1029, 13, 104, 88, 4, 381, 15, 297, 98, 32, 2071, 56, 26,
141, 6, 194, 7486, 18, 4, 226, 22, 21, 134, 476, 26, 480, 5, 144, 30, 5535, 18, 51, 36,
28, 224, 92, 25, 104, 4, 226, 65, 16, 38, 1334, 88, 12, 16, 283, 5, 16, 4472, 113, 103,
32, 15, 16, 5345, 19, 178, 32]
Train label 0 =  1
```

Each number corresponds to the corresponding English word.

1: Positive
0: Negative

Machine Translated by Google

Interpret numbers into words

- Word Index

```
word_index = imdb.get_word_index()
print(word_index['the'], word_index['and'], word_index['a'])
```

```
1 2 3
```

– Word number in IMDB dataset = word_index + 3

- 1~3 are assigned with special meanings (1: Begin of Sentence, 2: Unknown word, etc.) • e.g.) the = 4, and =

5, a = 6 – e.g.) I love you = [10, 116, 22]

- Code that interprets numbers into words

```
numToWord = {}
numToWord[0]=''
numToWord[1]='<시작>'
numToWord[2]='<미등록>'
numToWord[3]=''
for (word, num) in word_index.items():
    numToWord[num+3] = word
print(numToWord[4], numToWord[5], numToWord[6])

for i in train_data[0]: print(numToWord[i], end=' ')
```

```
the and a
```

```
<시작> this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you could just imagine being there robert redford's is an amazing actor and now the same being director norman's father came from the same scottish island as myself so i loved the fact there was a real connection with this film the witty remarks throughout the film were great it was just brilliant so much that i bought the film as soon as it was released for retail and would recommend it to everyone to watch and the fly fishing was amazing really cried at the end it was so sad and you know what they say if you cry at a film it must have been good and this definitely was also congratulations to the two little boy's that played the part's of norman and paul they were just brilliant children are often left out of the praising list i think because the stars that play them all grown up are such a big profile for the whole film but these children are amazing and should be praised for what they have done don't you think the whole story was so lovely because it was true and was someone's life after all that was shared with us all
```

Machine Translated by Google

Vectorization of review sentences

- Vectorize review sentences into a BOW model
  - Fix the word count to 1,000, consider only the 1,000 high-frequency words, and ignore subsequent words.

```
def vectorize(review_data):
    vec=np.zeros(1000, dtype='float32')
    for x in review_data:
        if x>=1000: continue
        vec[x]+=1
    return vec
```

  - Example of converting a specific review to BOW

```
print(vectorize(train_data[0]))
```

```
[ 0.  1.  0.  0. 15.  9.  3.  2.  3.  1.  0.  0.  6.  3.  3.  4. 11.  3.
  3.  2.  0.  1.  6.  0.  0.  4.  3.  0.  2.  0.  1.  0.  3.  2.  0.  1.
  ...  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
```

- Confirm that word frequency can be a large value

Machine Translated by Google

Training data configuration

- The element values of the BOW vector need to be normalized to be between 0 and 1 – Dividing by the maximum value results in low-frequency values (e.g., 1, 2 occurrences) becoming too small – Instead of dividing by the maximum value, convert 0 to 0, 1 to 0.25, 2 to 0.5, 3 to 0.75, and 4 **or higher to 1.**

```
def vectorize(review_data):
    vec=np.zeros(1000, dtype='float32')
    for x in review_data:
        if x>=1000: continue
        if vec[x]<1: vec[x]+=0.25
    return vec
```

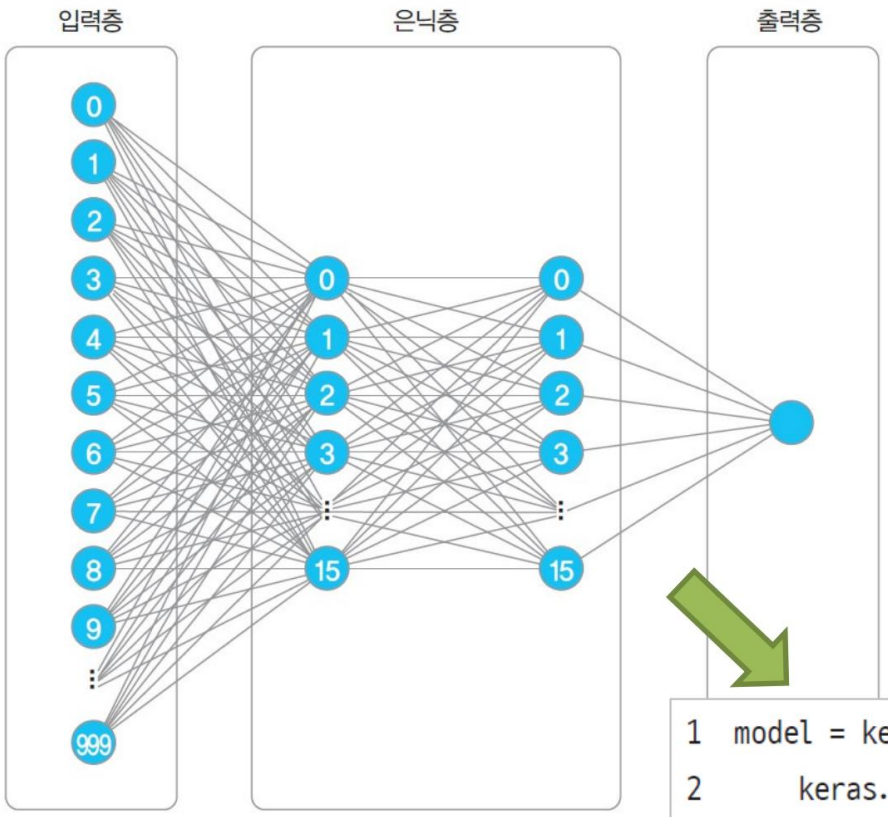
- Convert all training data to BOW

```
train_input = np.zeros((len(train_data), 1000))
for i, review in enumerate(train_data):
    train_input[i] = vectorize(review)
```

Machine Translated by Google

Neural network design

- 1,000 input nodes, 1 output neuron, and the hidden layer consists of 2 layers of 15 neurons each.



[그림 14-4] 감정분석 인공신경망 설계

```
model = keras.Sequential([
    keras.layers.Dense(16, activation='relu', input_shape=(1000,)),
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')])
```

Machine Translated by Google

Learning and prediction

- Setting up the optimization algorithm and loss function for training

```
model.compile(optimizer='adam', loss='binary_crossentropy')
```

- Perform training using training data and correct answers

```
model.fit(train_input, train_labels, epochs=3)
```

- Convert test data into a format suitable for neural network input

```
test_input = np.zeros((len(test_data), 1000))
for i, review in enumerate(test_data):
    test_input[i]=vectorize(review)
```

- Positive/Negative prediction for test data

```
pred = model.predict(test_input)
print(pred)
```

  – Since it is derived as a probability value between 0 and 1, a judgment of positive/negative must be made based on 0.5.

Machine Translated by Google

Positive/Negative Prediction and Performance Evaluation

- Based on an output value of 0.5, it is judged as positive if it is 0.5 or higher, and negative otherwise.

```
pred_labels=[]
for v in pred:
    if v>=0.5: pred_labels.append(1)
    else: pred_labels.append(0)
```

- Performance Metrics Output –

The accuracy of distinguishing between positive and negative was found to be approximately 87%, confirming significant performance even in a simple model.

```
from sklearn.metrics import classification_report
print(classification_report(test_labels, pred_labels))from sklearn.metrics import
```

```
              precision    recall  f1-score   support

           0       0.88      0.85      0.86     12500
           1       0.86      0.88      0.87     12500

    accuracy                           0.87     25000
   macro avg       0.87      0.87      0.87     25000
weighted avg       0.87      0.87      0.87     25000
```

Full code: ex14-5.py

Machine Translated by Google

Advancement of natural language processing technology

Machine Translated by Google

Limitations of BOW

- Words with the same frequency cannot have the same meaning – “A just-right day” vs. “A just-right day” • Order carries meaning

- Language model
  – Assigning probability to word order • e.g.) The probability that the sentence “walking along a mountain path full of blooming spring flowers” will appear is much higher than “walking along the mountain path full of blooming spring flowers” – e.g.) Search term recommendation

[그림 14–5] 구글 검색엔진에서의 검색어 추천

Machine Translated by Google

Development of word vectorization methods

- One-hot encoding
  - As the simplest method, number the words and define a vector of length equal to the number of words, converting each word into a vector where only the digit at the corresponding number is 1 and the rest are 0.
  - Too many zeros and no connection between words is expressed

- Word embedding – Placing words in an n-dimensional coordinate space based on semantic similarity •
  Word2Vec, FastText, GloVe, ELMo, etc.

- Word2Vec (Word-to-Vector) – A type of word embedding method announced by Google
  - Learning that uses neural networks to predict adjacent words from a specific word (skip-gram) or predict a specific word from adjacent words (CBOW: Continuous Bag-Of-Words).
  - Use the hidden layer output of the neural network as the coordinates of that word

- Methods for vectorizing documents or sentences have also advanced.
  - Doc2Vec (Document-to-Vector), Sent2Vec (Sentence-to-Vector), etc. – Used in various fields such as chatbots

Machine Translated by Google

Advancement of sentence context understanding technology

• Seq2Seq (Sequence-to-Sequence) Model – Used in machine translation, etc. • When each word of the English sentence “I am a student” is fed sequentially into a recurrent neural network such as LSTM or GRU, the network internally retains the context of this sentence in the hidden layer. • Along with the context vector, the Korean word vector “ÿÿ” is given as input, and the next word A method in which “I am a student” is sequentially produced by predicting it – called an encoder-decoder architecture • Encoder: The part that creates the context vector • Decoder: The part that generates a new sentence from this

Transformer architecture

– Limitations of the Seq2Seq model • Translation performance degrades as sentences become longer due to the limited length of context vectors

– Introduction of attention vectors • Vectors indicating which words in the original sentence the decoder should primarily pay attention to at a given point when generating the next word, in order to improve the limitations of the Seq2Seq model – Emergence of Transformer architecture

• Removes LSTM while keeping only the attention vector

• Enables training on large volumes of text through improved training speed and quality

Machine Translated by Google

Development of large-scale language models

- A language model based on Transformer architecture that has been pre-trained on a large amount of text
  - GPT (Generative Pre-trained Transformer) • Announced by the OpenAI Foundation • Expanded to GPT (2018), GPT-2 (2019), and GPT-3 (2020)
  - BERT (Bidirectional Encoder Representations from Transformers) • Released by Google in 2018
  - Korean Version • Released by SKT, Kakao, Naver, etc.